

Limited Perfect Monotonical Surrogates constructed using low-cost recursive linkage discovery with guaranteed output

Michał W. Przewozniczek
Wrocław Univ. of Science and Techn.
Wrocław, Poland
michal.przewozniczek@pwr.edu.pl

Renato Tinós
University of São Paulo
Ribeirão Preto, Brazil
rtinos@ffclrp.usp.br

Francisco Chicano
ITIS Software, University of Málaga
Málaga, Spain
chicano@uma.es

Marcin M. Komarnicki
Wrocław Univ. of Science and Techn.
Wrocław, Poland
marcin.komarnicki@pwr.edu.pl

Abstract

Surrogates provide a cheap solution evaluation and offer significant leverage for optimizing computationally expensive problems. Usually, surrogates only approximate the original function. Recently, the perfect linear surrogates were proposed that ideally represent the original function. These surrogates do not mimic the original function. In fact, they are another (correct) representation of it and enable a wide range of possibilities, e.g., discovering the optimized function for problems where the direct transformation of the encoded solution into its evaluation is not available. However, many real-world problems can not be represented by linear models, making the aforementioned surrogates inapplicable. Therefore, we propose the Limited Monotonical Perfect Surrogate (LyMPuS), which overcomes this difficulty and enables the comparison of two solutions that differ by a single variable. Our proposition is suitable for limiting the cost of expensive local search procedures. The proposed surrogate is parameterless and can be trained on the fly without any separate surrogate-building step. It uses only the necessary fitness evaluations, and the already-paid costs are not wasted when the model is updated. Finally, it offers low-cost missing-linkage detection and low-cost linkage discovery, guaranteed to find a missing dependency in no more than $2\lceil\log_2(n)\rceil$ steps.

CCS Concepts

• **Mathematics of computing** → **Combinatorial optimization**;
• **Theory of computation** → **Random search heuristics**; • **Computing methodologies** → **Artificial intelligence**.

Keywords

Variable dependency, Perfect surrogates, Genetic Algorithms, Linkage learning

ACM Reference Format:

Michał W. Przewozniczek, Francisco Chicano, Renato Tinós, and Marcin M. Komarnicki. 2026. Limited Perfect Monotonical Surrogates constructed

using low-cost recursive linkage discovery with guaranteed output. In *Genetic and Evolutionary Computation Conference (GECCO '26)*, July 13–17, 2026, San Jose, Costa Rica. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3795095.3805171>

1 Introduction

The evaluation of many real-world problems is expensive and surrogates can significantly leverage the effectiveness and efficiency of the optimizers applied to solve them [27]. Typically, surrogates aim at approximating the optimized function [9]. Recently, perfect surrogates were proposed for the binary [5, 28] and permutation-based problems [3]. Obviously, perfect surrogates are equivalent to evaluating the original function and have a significant advantage over the typical surrogates that only approximate solution evaluation. However, their capabilities are significantly broader. Many real-world problems require indirect solution encoding; e.g., a simulation must be executed to evaluate the solution, or the encoding serves as input to the solution-building algorithm [1, 34]. In such cases, we know the exact function that evaluates the final solution, e.g., a production plan. However, we usually do not know the function that is optimized, i.e., one that directly transforms the encoded solution into its evaluation. Perfect surrogates allow us to overcome this difficulty and discover an optimized function [27].

Perfect surrogates require the non-linearity check to identify the variable dependencies needed to build their internal function model. For many problems, the non-linearity check is oversensitive [15], making the construction of a perfect surrogate intractable. Recent research shows that the non-linearity check is inappropriate for decomposing many real-world problems, which require using the non-monotonicity check [16]. Finally, variable dependency discovery, the first step of the perfect surrogate building procedure, often resembles a blind-luck search and must be performed before surrogate training, making the whole process cumbersome [5].

To fill the aforementioned gaps, we propose the Limited Monotonical Perfect Surrogate (LyMPuS) that (if trained on the complete set of dependencies obtained using the non-monotonicity check) is guaranteed to compare the quality of two solutions differing by one variable. Such a functionality is suitable to leverage expensive local search procedures. The proposed surrogate is flexible; it is trained on the fly, using fitness evaluations only when necessary, and avoids wasting any of the already-paid costs. It supports low-cost missing linkage detection and the low-cost linkage discovery



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

GECCO '26, San Jose, Costa Rica

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2487-9/2026/07

<https://doi.org/10.1145/3795095.3805171>

that is guaranteed to find one missing dependency in no more than $2\lceil \log_2(n) \rceil$ steps, where n is the problem size.

2 Background

In many combinatorial problems some subsets of variables may have a joint non-linear or non-monotonical influence on function value and, therefore, should be processed jointly [16, 29]. Here, we consider the maximization of $f(\mathbf{x})$, where $\mathbf{x} = [x_1, x_2, \dots, x_n]$ is a binary solution vector of size n . However, the mechanisms proposed in this work apply to other search spaces as well.

Example 2.1. Consider $f(x_1, \dots, x_8) = f_1(x_1, \dots, x_4) + f_2(x_5, \dots, x_8)$, where each $f_i(x_{4i-3}, \dots, x_{4i}) = \text{bim}_k(u(x_{4i-3}, \dots, x_{4i}))$, where bim_k is bimodal deceptive function or order k [4]:

$$\text{bim}_k(u(\mathbf{x})) = \begin{cases} k/2 - |u - k/2| - 1 & , u \neq k \wedge u \neq 0 \\ k/2 & , u = k \vee u = 0 \end{cases} \quad (1)$$

where u is the sum of gene values (so-called *unitation*).

Intuitively variables in groups $\{x_1, \dots, x_4\}$ and $\{x_5, \dots, x_8\}$ depend on each other and each group can be processed separately to find the optimal solution.

We can identify variable dependencies by checking the results of variable perturbation. We define $\Delta_g(\mathbf{x}) = f(\mathbf{x}^g) - f(\mathbf{x})$ and $\Delta_{g,h}(\mathbf{x}) = f(\mathbf{x}^{g,h}) - f(\mathbf{x})$, where $\mathbf{x}^g$ equals $\mathbf{x}$ with x_g flipped, and $\mathbf{x}^{g,h}$ equals $\mathbf{x}$ with x_g and x_h flipped. If, for a given $\mathbf{x}$, flipping x_g and x_h causes fitness change of Δ_g and Δ_h , respectively, then flipping them both should modify fitness by $\Delta_{g,h}$. Otherwise, x_g and x_h are *non-linearly* dependent. In the *non-linearity check* [14], x_g and x_h are dependent if: $f(\mathbf{x}) + f(\mathbf{x}^{g,h}) \neq f(\mathbf{x}^g) + f(\mathbf{x}^h)$. In Example 2.1, in two separate groups, $\{x_1, \dots, x_4\}$ and $\{x_5, \dots, x_8\}$, variables are non-linearly dependent on each other.

Any problem can be represented in the *additive form*: $f(\mathbf{x}) = \sum_{s=1}^S f_s(\mathbf{x}_{I_s})$, where the argument set $\mathbf{x}_{I_s}$ of each subfunction f_s is a subset of $\mathbf{x}$, and $\mathbf{x}_{I_s}$ do not have to be disjoint. The non-linearity check is convenient for decomposing the k -bounded problems that can be represented in such an additive form that each subfunction takes no more than k arguments.

The non-linearity check is oversensitive for many real-world problems [15, 16]. Using $f(\dots)$ from Example 2.1, we can define $f'(x_1, \dots, x_8) = (f(x_1, \dots, x_8))^2$. For any two solutions $\mathbf{x}_a$ and $\mathbf{x}_b$, it is true that if $f(\mathbf{x}_a)Rf(\mathbf{x}_b)$, then $f'(\mathbf{x}_a)Rf'(\mathbf{x}_b)$ where $R \in \{>, <=, =\}$. Thus, f and f' have the same local and global optima and their attraction basins. However, the non-linearity check will recognize all variables in f' as dependent on each other, which is useless information. To this end, the non-monotonicity check was proposed [15]. We present its slightly more sensitive version [17], i.e., x_g and x_h are non-monotonically dependent if at least one clause holds:

C1. $f(\mathbf{x}) < f(\mathbf{x}^g) \wedge f(\mathbf{x}^h) \geq f(\mathbf{x}^{g,h})$ C4. $f(\mathbf{x}) < f(\mathbf{x}^h) \wedge f(\mathbf{x}^g) \geq f(\mathbf{x}^{g,h})$
C2. $f(\mathbf{x}) = f(\mathbf{x}^g) \wedge f(\mathbf{x}^h) \neq f(\mathbf{x}^{g,h})$ C5. $f(\mathbf{x}) = f(\mathbf{x}^h) \wedge f(\mathbf{x}^g) \neq f(\mathbf{x}^{g,h})$
C3. $f(\mathbf{x}) > f(\mathbf{x}^g) \wedge f(\mathbf{x}^h) \leq f(\mathbf{x}^{g,h})$ C6. $f(\mathbf{x}) > f(\mathbf{x}^h) \wedge f(\mathbf{x}^g) \leq f(\mathbf{x}^{g,h})$

Consider clause C1 – flipping x_g improves fitness. However, if x_h is flipped, flipping x_g does not improve fitness anymore. Thus, x_g depends on x_h . Some recent studies show that the non-monotonicity check can discover non-symmetrical relations (i.e., x_g depends on x_h but x_h does not have to depend on x_g) and that telling them from

symmetrical relations can cause a significant performance improvement while optimizing some real-world problems [17]. Here, we focus on symmetrical dependencies, but all proposed mechanisms also apply to discovering non-symmetrical dependencies.

In complex real-world problems, the subfunction argument sets frequently overlap, i.e., share variables [2, 16]. Such overlapping problems are hard to optimize because, in the general case, subfunctions can not be optimized separately. Gray-box optimization, where the variable dependencies are known *a priori*, proposes effective operators to handle overlapping problems [29]. A variable dependency network is frequently represented by the Variable Interaction Graph (VIG, G). G contains a complete set of true dependencies of a given type, e.g., non-linear (G_{NL}) or non-monotonical (G_{NM}). For any $\mathbf{x}$ and x_g , using G (of any type), one can construct a perturbation mask m containing (all or a subset of) variables dependent on x_g . After perturbing, $\mathbf{x}$ is the subject of local optimization yielding $\mathbf{x}'$. If $f(\mathbf{x}') \geq f(\mathbf{x})$, then $\mathbf{x}'$ replaces $\mathbf{x}$ or is rejected otherwise. Such simple, yet dependency-driven, operators were shown to be effective in solving many complex overlapping problems [24].

Partition Crossover (PX) is a gray-box mixing operator [25]. For two mixed individuals, say $\mathbf{x}_a$ and $\mathbf{x}_b$, it clusters all variables differing in $\mathbf{x}_a$ and $\mathbf{x}_b$ with respect to G . PX was shown to produce mixing masks that preserve subfunction argument sets even if they overlap [18]. For G_{NL} , any mask yielded by PX is proven to yield offspring such that $f(\mathbf{x}_a) + f(\mathbf{x}_b) = f(\mathbf{x}_{oa}) + f(\mathbf{x}_{ob})$ [31]. For G_{NM} , PX is proven to yield offspring such that:

- if $f(\mathbf{x}_a)R_{ine}f(\mathbf{x}_{oa})$ then $f(\mathbf{x}_b)\neg R_{ine}f(\mathbf{x}_{ob})$, where $R_{ine} = \{>, <\}$, $\mathbf{x}_{oa} = \mathbf{x}_a \xleftarrow{mPX} \mathbf{x}_b$ ($\mathbf{x}_{oa}$ is a copy of $\mathbf{x}_a$ with gene values marked by mask mPX copied from $\mathbf{x}_b$), and mPX is any PX mask yielded for $\mathbf{x}_a$ and $\mathbf{x}_b$.
- if $f(\mathbf{x}_a) = f(\mathbf{x}_{oa})$ then $f(\mathbf{x}_b) = f(\mathbf{x}_{ob})$.

In black-box optimization, variable dependencies are unknown. Among all, we can discover them using non-linearity or non-monotonicity checks, store them in the empirical VIG (eG), and use them in the gray-box manner. All dependencies discovered using such checks are true. Thus, $eG \subseteq G$. PX can be used for individual mixing and missing linkage detection, i.e., when the offspring does not fulfill the aforementioned features, then we know the PX mask is incomplete. Thus, there must exist at least one variable from the PX mask and one from the rest of the genotype that are dependent [18]. See Section S-I (supplementary material) for PX examples.

Any pseudo-Boolean function can be represented as $f(\mathbf{x}) = \sum_{i=0}^{2^n-1} w_i \varphi_i(\mathbf{x})$ where $w_i \in \mathbb{R}$ is the i th Walsh coefficient, $\varphi_i(\mathbf{x}) = (-1)^{\mathbf{i}^T \mathbf{x}}$ generates a sign, and $\mathbf{i} \in \{0, 1\}^n$ is the binary representation of index i . The Walsh-based model of $f(\mathbf{x})$ can be constructed in black-box optimization [3, 5, 12]. To this end, one have to discover non-linear dependencies and use the Walsh-based model construction algorithm [7]. If all non-linear dependencies were discovered, then the Walsh-based model would perfectly represent the optimized function, allowing for evaluating it using the obtained *Perfect Linear Surrogate* rather than the original function. Such surrogates enable much more than just free fitness evaluations. For instance, when $\mathbf{x}$ defines a entry information to some solution-building algorithm, e.g., in feature selection problems [16], then we frequently do not know the function that directly transforms $\mathbf{x}$ into its evaluation (in feature selection problems we usually know a function that

Table 1: G_{NM} and LyMPuS for f_{e1} .

	x_1	x_2	x_3	x_4
x_1	X	1	0	0
x_2	1	X	1	0
x_3	0	1	X	1
x_4	0	0	1	X

(a) G_{NM} for Example 3.1

x_g	$C_{G_{NM}}^g$	$C_{G_{NM}}^g(x)$	$b(x_g, C_G^g(x), x)$
x_1	$\{x_2\}$	0^{**}	$\{1\}$
x_2	$\{x_1, x_3\}$	0^*0^*	$\{1\}$
		0^*1^*	$\{0, 1\}$
		1^*0^*	$\{0, 1\}$
		1^*1^*	$\{0\}$
x_3	$\{x_2, x_4\}$	$^*0^*0$	$\{1\}$
		$^*0^*1$	$\{0, 1\}$
		$^*1^*0$	$\{0, 1\}$
		$^*1^*1$	$\{0\}$
x_4	$\{x_3\}$	$^{**}0^*$	$\{1\}$
		$^{**}1^*$	$\{0\}$

(b) LyMPuS for Example 3.1

evaluates an encoded classifier but not a function that evaluates the encoding). Perfect Surrogates allow to break this limitation.

Both non-linearity and non-monotonicity checks are applicable for continuous problems as well [10, 13, 21]. Flipping a variable value is then equivalent to setting a different value within its domain. Problems are mainly first decomposed, and each part is optimized separately afterward. In this approach, parts are disjoint, which may not be suitable for overlapping problems [11]. To reduce the decomposition cost, recursive decomposition strategies are used [10, 13, 21]. Most of them group all variables together when overlapping problems are considered [21]. Moreover, searching for dependencies for a single variable starts with the interaction check with all variables that do not yet belong to any group.

Here, we consider optimizers using Statistical Linkage Learning (SLL) [6, 8, 22] supported by the recently proposed First Improvement Hillclimber with Linkage Learning (FIHCwLL) [19]. Since these optimizers are only used to present the quality of our propositions, they are described in Section S-II (supplementary material).

3 Limited Monotonic Perfect Surrogate

3.1 Motivations and ideas

Assume that G is known. We define C_G^g , the *close context* of x_g as the set of all variables adjacent to x_g in G . By $C_G^g(x)$, we define a hyperplane [31] such that the values of C_G^g are taken from x . The set of variables marked by a hyperplane ϕ we define as $def(\phi)$. For instance, for $x = [111000]$ and $C_G^2 = \{x_1, x_3, x_6\}$, $C_G^2(x) = [1^*1^{**}0]$, where $*$ indicates the *don't care* value. Finally, $def([1^*1^{**}0]) = C_G^2$.

THEOREM 1. Consider any x_a, x_b such that x_g equals in x_a, x_b , $C_G^g(x_a) = C_G^g(x_b)$, and $R_1, R_2 \in \{<, =, >\}$ such that $f(x_a)R_1f(x_a^g)$, $f(x_b)R_2f(x_b^g)$. If $G = G_{NL}$ or $G = G_{NM}$, then $R_1 = R_2$.

PROOF. If $G = G_{NL}$, then $\Delta_g(x_a) \neq \Delta_g(x_b)$ if and only if at least one gene differing in x_a and x_b is non-linearly dependent on x_g . Since $C_{G_{NL}}^g(x_a) = C_{G_{NL}}^g(x_b)$, then all genes non-linearly dependent on x_g are equal in x_a and x_b . Thus, $\Delta_g(x_a) = \Delta_g(x_b)$ and $R_1 = R_2$.

If $G = G_{NM}$, then $R_1 \neq R_2$ if and only if one of the C1-C6 clauses holds for x_g and at least one other gene that differs in x_a and x_b . Since, $C_{G_{NM}}^g(x_a) = C_{G_{NM}}^g(x_b)$, then all genes for which at least one of C1-C6 clauses is satisfied for x_g are equal in x_a and x_b . Thus, $R_1 = R_2$. $\square$

For any x_g and a hyperplane ϕ such that $x_g \notin def(\phi)$ we define

$$b(x_g, \phi, x) = \begin{cases} \{0\} & \text{if } f(x|_{x_g=0, def(\phi)=\phi}) > f(x|_{x_g=1, def(\phi)=\phi}) \\ \{1\} & \text{if } f(x|_{x_g=0, def(\phi)=\phi}) < f(x|_{x_g=1, def(\phi)=\phi}) \\ \{0, 1\} & \text{if } f(x|_{x_g=0, def(\phi)=\phi}) = f(x|_{x_g=1, def(\phi)=\phi}) \end{cases} \quad (2)$$

where $x|_{x_g=v, def(\phi)=\phi}$ is a solution obtained from x by setting the value of x_g to v and the value of all $x_i \in def(\phi)$ to ϕ , e.g., for $\phi = [***111]$, $[111000]|_{x_3=0, def(\phi)=\phi} = [110111]$.

Note that for any $x, b(x_g, \cdot, x)$ (where $\cdot$ is a hyperplane with no defined positions, i.e., $[*****]$) indicates if flipping x_g is an improving, deteriorating or a sliding move for x .

From Theorem 1 it follows that for each x_g and a ϕ such that $def(\phi) = C_G^g$, it is true that $\forall x_a, x_b, b(x_g, \phi, x_a) = b(x_g, \phi, x_b)$. We define the Limited Monotonic Perfect Surrogate (LyMPuS) as

$$LyMPuS(x_g, G, x) = b(x_g, C_G^g(x), x) \quad (3)$$

If G is known, then to construct LyMPuS it is enough to collect and store the outputs of $b(x_g, C_G^g(x), x)$ for all ϕ such that $def(\phi) = C_G^g$ for each x_g . Then, we can use LyMPuS for checking if flipping x_g for any solution x improves, decreases or leaves its fitness unchanged. If LyMPuS is constructed for G_{NM} , it is convenient for problems that can not be handled by Walsh-based surrogates.

Example 3.1. Consider $f(x_1, \dots, x_4) = f_1(x_1, x_2) \cdot f_2(x_2, x_3) \cdot f_3(x_3, x_4)$, where $f_i(x_i, x_{i+1}) = xor(x_i, x_{i+1}) + 1$. Note that G_{NL} of f is a full graph. G_{NM} and LyMPuS for f are presented in Tables 1a and 1b, respectively. For each variable x_g , LyMPuS stores $2^{|C_{G_{NM}}^g|}$ hyperplanes to determine the output of $b(x_g, C_G^g(x), x)$ for any x (Table 1b, the last column). For instance, $b(x_1, C_G^1(x), x)$ depends only on x_2 , i.e., $b(x_1, ^*0^{**}, x) = \{1\}$ and $b(x_1, ^*1^{**}, x) = \{0\}$. In contrast, $b(x_2, C_G^2(x), x)$ depends on x_1 and x_3 and some of its outputs contain more than value, e.g., $b(x_2, 0^*1^*, x) = \{0, 1\}$ (in the 0^*1^* hyperplane, both values of x_2 yield the same fitness).

From this point onward, unless stated otherwise, **we assume G_{NM} and eG_{NM} whenever we refer to G and eG , respectively.** In the black-box scenario, G is not available but we can use eG instead and construct the *empirical* LyMPuS (eLyMPuS), i.e., LyMPuS constructed for eG . To determine any $b(x_g, C_{eG}^g(x), x)$, we have to compute $b(x_g, \cdot, x)$ which equals $b(x_g, C_G^g(x), x)$. Thus, even if C_{eG}^g is incomplete in a given moment, the value of $b(x_g, \cdot, x)$ will remain valid after C_{eG}^g is supplemented, i.e., no fitness evaluations spared for constructing eLyMPuS will be wasted.

Walsh-based models enable *partial evaluations*, i.e., re-evaluating only a part of the model when only a few variables were modified. Analogously, LyMPuS allows comparing solutions that differ by one gene concerning only a part of their genotypes, i.e., C_G^g . Therefore, we call this functionality *partial comparisons*.

3.2 Partial comparisons, guaranteed linkage discovery, and eLyMPuS construction

Many surrogate-assisted optimizers (including those using Perfect Linear Surrogates) are divided into two main steps, i.e., the surrogate-building step, and the surrogate-using (pure optimization) step [5, 27]. However, in some cases, to find a solution of satisfactory quality, the surrogate does not have to model the entire

solution space perfectly. Therefore, the aforementioned separation of the surrogate-building and surrogate-using steps can be considered inflexible and less effective than updating the surrogate iteratively as a part of the optimization process. The same observation applies to modeling variable dependencies [23].

eLyMPuS is dedicated to black-box scenarios and is constructed on the fly. It will evaluate $f(\mathbf{x})$ only in three situations: to verify its correctness, to discover missing variable dependencies, or if $b(x_g, C_{eG}^g(\mathbf{x}), \mathbf{x})$ was not computed for a given $C_{eG}^g(\mathbf{x})$ yet.

Pseudocode 1 eLyMPuS Initialization

```

1: procedure INITIALIZE(eLyMPuS)
2:   eLyMPuS.eG  $\leftarrow$  empty;
3:   for each  $x_g$  in  $\{x_1, \dots, x_n\}$  do
4:     eLyMPuS.x_gPairs( $\mathbf{x}, b_g^*(\mathbf{x})$ )  $\leftarrow$  empty;

```

Pseudocode 1 presents the eLyMPuS initialization procedure executed once at the start of the eLyMPuS-using optimizer. For each x_g , eLyMPuS maintains a set of initially empty pairs $(\mathbf{x}, b_g^*(\mathbf{x}))$, where $b_g^*(\mathbf{x}) = b(x_g, \cdot, \mathbf{x})$.

Pseudocode 2 LyMPuS partial comparison procedure

```

1: function PARTIALCOMPARISON( $x_g, \mathbf{x}_{chk}, eLyMPuS, Verify$ )
2:   Pair  $\leftarrow$  FindPair( $x_g, \mathbf{x}_{chk}, eLyMPuS$ ); ▷ line 14
3:   if Pair  $\neq$  empty then
4:     if Verify = true then
5:        $b_{g,chk}^* \leftarrow$  Compute_ $b_g^*$ ( $x_g, \mathbf{x}_{chk}$ ); ▷ line 20
6:       if Pair. $b_g^* \neq b_{g,chk}^*$  then
7:         RecursiveLL(Pair. $\mathbf{x}, \mathbf{x}_{chk}, x_g, eLyMPuS$ );
8:         AddNewPair(eLyMPuS.x_gPairs( $\mathbf{x}, b_g^*(\mathbf{x})$ ),
9:           Pair( $\mathbf{x}_{chk}, b_{g,chk}^*$ ));
10:        return  $b_{g,chk}^*$ ;
11:      return Pair. $b_g^*$ ;
12:     $b_{g,res}^* \leftarrow$  Compute_ $b_g^*$ ( $x_g, \mathbf{x}_{chk}$ ); ▷ line 20
13:    AddNewPair(eLyMPuS.x_gPairs( $\mathbf{x}, b_g^*(\mathbf{x})$ ), Pair( $\mathbf{x}_{chk}, b_{g,res}^*$ ));
14:    return  $b_{g,res}^*$ ;

14: function FINDPAIR( $x_g, \mathbf{x}_{chk}, eLyMPuS$ )
15:   eG  $\leftarrow$  eLyMPuS.eG;
16:   for each Pair( $\mathbf{x}, b_g^*(\mathbf{x})$ ) in eLyMPuS.x_gPairs( $\mathbf{x}, b_g^*(\mathbf{x})$ ) do
17:     if  $C_{eG}^g(Pair.\mathbf{x}) = C_{eG}^g(\mathbf{x}_{chk})$  then
18:       return Pair;
19:   return empty;

20: function COMPUTE_ $b_g^*$ ( $x_g, \mathbf{x}_{chk}$ )
21:   fit  $\leftarrow$  fit( $\mathbf{x}_{chk}$ );
22:   fit $_g \leftarrow$  fit( $\mathbf{x}_{chk}^g$ );
23:   if fit < fit $_g$  then  $b_g^* \leftarrow \{\mathbf{x}_{chk}^g[g]\}$ 
24:   else
25:     if fit > fit $_g$  then  $b_g^* \leftarrow \{\mathbf{x}_{chk}[g]\}$ 
26:     else  $b_g^* \leftarrow \{0, 1\}$ 
27:   return  $b_g^*$ ;

```

In the black-box scenarios, we never know if $C_{eG}^g = C_G^g$. Therefore, for each x_g , we store a list of pairs $(\mathbf{x}, b_g^*(\mathbf{x}))$. The list is supplemented in subsequent partial comparison executions for such

x_g and $\mathbf{x}_{chk}$ that $C_{eG}^g(\mathbf{x}_{chk})$ does not exist in the list for a given x_g . If new dependencies are found during the run and C_{eG}^g is supplemented, all pairs in the list remain useful, i.e., the already paid cost of building the model is never wasted.

During partial comparison (Pseudocode 2) for $\mathbf{x}_{chk}$ and x_g , eLyMPuS searches the list of pairs $(\mathbf{x}, b_g^*(\mathbf{x}))$ (line 2). If it finds a pair such that $C_{eG}^g(Pair.\mathbf{x}) = C_{eG}^g(\mathbf{x}_{chk})$, i.e., for all variables dependent on x_g in G , their values in *Pair*. $\mathbf{x}$ and $\mathbf{x}_{chk}$ are equal, then *Pair*. b_g^* is a result (line 10). If the appropriate pair does not exist, $b_g^*(\mathbf{x}_{chk})$ is computed and a new pair is added to the list (lines 11–12).

eLyMPuS uses *eG* that may be incomplete. Therefore, sometimes it should verify if linkage is not missing. In such a case, $b_g^*(\mathbf{x}_{chk})$ is computed (line 5). If *eG* contains all dependencies for x_g , i.e., $C_{eG}^g = C_G^g$, then it is guaranteed that $b_g^*(\mathbf{x}_{chk}) = b(x_g, C_{eG}^g(\mathbf{x}_{chk}), Pair.\mathbf{x})$. In the opposite situation, the results may differ (line 6). Then, we **know** that differing results are caused by the missing dependency between x_g and at least one variable that differs in $\mathbf{x}_{chk}$ and *Pair*. $\mathbf{x}$.

Pseudocode 3 Guaranteed Linkage Discovery with Recursive LL

```

1: procedure RECURSIVELL( $\mathbf{x}_{s1}, \mathbf{x}_{s2}, x_g, eLyMPuS$ )
2:   eG  $\leftarrow$  eLyMPuS.eG;
3:   GenesToConsider  $\leftarrow$  Diff( $\mathbf{x}_{s1}, \mathbf{x}_{s2}$ ) \  $x_g$ ;
4:   RecForGroup( $\mathbf{x}_{s1}, \mathbf{x}_{s2}, x_g, GenesToConsider, eLyMPuS$ );

5: procedure RECFORGROUP( $\mathbf{x}_{s1}, \mathbf{x}_{s2}, x_g, GroupToDiv, eLyMPuS$ )
6:   if |GroupToDiv| = 1 then
7:     eLyMPuS.eG[ $x_g$ ][GroupToDiv[1]]  $\leftarrow$  1;
8:     eLyMPuS.eG[GroupToDiv[1]][ $x_g$ ]  $\leftarrow$  1;
9:     return;
10:     $b_{g,s2}^* \leftarrow$  Compute_ $b_g^*$ ( $x_g, \mathbf{x}_{s2}$ ); ▷ Pseudocode 2, line 20
11:     $gr_A, gr_B \leftarrow$  RandDivideInTwoEqualGroups (GroupToDiv);
12:     $\mathbf{x}'_{s1} \leftarrow \mathbf{x}_{s1}; \mathbf{x}'_{s1} \leftarrow_{gr_A} \mathbf{x}_{s2}$ ;
13:     $\mathbf{x}^*_{g,s'1} \leftarrow$  Compute_ $b_g^*$ ( $x_g, \mathbf{x}'_{s1}$ );
14:    if  $b_{g,s'1}^* = b_{g,s2}^*$  then
15:      RecForGroup( $\mathbf{x}_{s1}, \mathbf{x}'_{s1}, x_g, gr_A, eLyMPuS$ );
16:    else
17:      RecForGroup( $\mathbf{x}'_{s1}, \mathbf{x}_{s2}, x_g, gr_B, eLyMPuS$ );

```

The above procedure finds two solutions (*Pair*. $\mathbf{x}$ and $\mathbf{x}_{chk}$) for which $b_g^*(\mathbf{x}_{chk}) \neq b_g^*(Pair.\mathbf{x})$, although $C_{eG}^g(\mathbf{x}_{chk}) = C_{eG}^g(Pair.\mathbf{x})$. Thus, $C_{eG}^g \subset C_G^g$. For the sake of clarity, in Pseudocode 3, we denote these two solutions as $\mathbf{x}_{s1}, \mathbf{x}_{s2}$. Let $S = \{\mathbf{x}_0, \dots, \mathbf{x}_{|C|}\}$ be a sequence of solutions such that:

- (1) $C = \text{Diff}(\mathbf{x}_{s1}, \mathbf{x}_{s2}) \setminus x_g$, where $\text{Diff}(\mathbf{x}_{s1}, \mathbf{x}_{s2})$ is the set of all variables that differ in $\mathbf{x}_{s1}, \mathbf{x}_{s2}$ (note that $C_{eG}^g(\mathbf{x}_{s1}) = C_{eG}^g(\mathbf{x}_{s2})$).
- (2) $\mathbf{x}_0 = \mathbf{x}_{s1}$
- (3) For each $x_{c_i} \in C$, considered in any order, $\mathbf{x}_i = \mathbf{x}_{i-1} \xleftarrow{x_{c_i}} \mathbf{x}_{s2}$, where $\mathbf{x}_a \xleftarrow[m]{\mathbf{x}_b}$ denotes copying variable values marked by a mask m from $\mathbf{x}_b$ to $\mathbf{x}_a$. Note that $\mathbf{x}_i = \mathbf{x}_{i-1}^{x_{c_i}}$, i.e., $\mathbf{x}_{i-1}$ with x_{c_i} flipped.

THEOREM 2. Sequence S contains two consecutive solutions $\mathbf{x}_{i-1}, \mathbf{x}_i$ such that $\mathbf{x}_{i-1}, \mathbf{x}_{i-1}^g, \mathbf{x}_{i-1}^{c_i}, \mathbf{x}_{i-1}^{g \cdot c_i}$ fulfill C1, C2 or C3 clause.

PROOF. The definition of S , states that $b_g^*(\mathbf{x}_0) = b_g^*(\mathbf{x}_{s1}) \neq b_g^*(\mathbf{x}_{s2}) = b_g^*(\mathbf{x}_{|C|})$. All neighbouring solutions in S differ by one

Table 2: eG discovered by eLyMPuS in Example 3.2 (missing dependencies are marked in red)

	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8
x_1	X	1	0	0	0	0	0	0
x_2	1	X	0	0	0	0	0	0
x_3	0	0	X	1	0	0	0	0
x_4	0	0	1	X	0	0	0	0
x_5	0	0	0	0	X	1	0	0
x_6	0	0	0	0	1	X	0	0
x_7	0	0	0	0	0	0	X	1
x_8	0	0	0	0	0	0	1	X

Table 3: eLyMPuS for Example 3.2

x_g	C_{eG}^g	$Pair.x$	$C_{eG}^g(Pair.x)$	$b_g^*(Pair.x)$
x_1	$\{x_2\}$	01 10 10 01	*1 ** ** *	$\{0\}$
		10 10 10 01	*0 ** ** *	$\{1\}$
x_2	$\{x_1\}$	10 10 10 01	1* ** ** *	$\{0\}$
x_3	$\{x_4\}$	11 11 00 11	** 1* ** *	$\{1\}$
		00 00 11 00	** 0* ** *	$\{0\}$
x_4	$\{x_3\}$	01 10 01 10	** 1* ** *	$\{0\}$
		10 01 01 10	** 0* ** *	$\{1\}$
x_5	$\{x_6\}$	01 10 10 01	** ** 0* **	$\{1\}$
x_6	$\{x_5\}$	01 10 10 01	** ** 1* **	$\{0\}$
x_7	$\{x_8\}$	01 10 10 01	** ** ** 1*	$\{0\}$
x_8	$\{x_7\}$	01 10 10 01	** ** ** 0*	$\{1\}$

gene. Thus, S contains consecutive solutions, x_{i-1}, x_i such that $b_g^*(x_{i-1}) \neq b_g^*(x_i)$, therefore, $f(x_{i-1})R_{i-1}f(x_{i-1}^g), f(x_i)R_i f(x_i^g)$, and $R_{i-1} \neq R_i$. Since $x_i = x_{i-1}^{c_i}$, then $f(x_{i-1}^{c_i})R_i f(x_i^{g,c_i})$, which proves that using x_{i-1}, x_g , and x_{c_i} must fulfil one of C1-C3 clauses (for each clause, $h = c_i$). $\square$

The above theorem proves that when PARTIALCOMPARISON finds missing linkage for solutions x_{chk} and $Pair.x$, then we are *guaranteed* to find one dependency that is missing in $eLyMPuS.eG$. Moreover, we are only guaranteed to fill one of the C1-C3 clauses (not the C4-C6). Thus, we can distinguish directional and symmetrical dependencies. The dependency will be discovered in no more than $|C|$ steps. If we perform this operation recursively and split C into two equal (or almost equal) groups in each step, we are guaranteed to find one missing dependency in $\lceil \log_2(|C|) \rceil$ steps.

Pseudocode 3 presents the recursive linkage discovery version. Initially, we consider all genes in C (line 3). In each step, we split $GenesToConsider$ into two groups (gr_A, gr_B) of equal size (or sizes differing by one if $|GenesToConsider|$ is an odd number). We create x'_{s1} , a copy of x_{s1} , and the genes from x_{s2} marked by gr_A . If $b_g^*(x'_{s1}) = b_g^*(x'_{s2})$, we split gr_A (line 15). Otherwise, we split gr_B for solutions x'_{s1} and x_{s2} (line 17). When $|GenesToConsider| = 1$, we are guaranteed that we have found two solutions that differ by one gene for which flipping x_g yields different $b_g^*(x)$.

Example 3.2. Consider $f(x_1, \dots, x_8) = f_1(x_1, \dots, x_4) + f_2(x_3, \dots, x_6) + f_3(x_5, \dots, x_8)$, where $f_i = bim_4(x_{2i-1}, \dots, x_{2i+2})$. We use eLyMPuS from Table 3 to locally optimize $x = [11 10 01 01]$; $f(x) = f_1(1110) + f_2(1001) + f_3(0101) = 0 + 1 + 1 = 2$. We try to perform bit-flips in the gene order. We use partial comparisons and for each such operation, we verify if the eLyMPuS result is correct. For x_1 , $C_{eG}^1(Pair.01101001) = C_{eG}^1(x) = [1* ** ** *]$, and eLyMPuS returns $\{0\}$. We compute $b_1^*(x) = \{0\}$ that confirms eLyMPuS answer (Pseudocode 2, line 5). Solution x is modified to $x^1 = [01 10 01 01]$.

Next, we try to flip x_2 in x^1 . However, $C_{eG}^2(x^1) = 0* ** ** *$, and there is no solution in the list of pairs $(x, b_g^*(x))$ that would fit

to it. Therefore, we compute $b_2^*(x^1) = \{1\}$, and add the new pair $(x^1, b_2^*(x^1))$ to the list of pairs for x_2 (Pseudocode 2, line 12).

In the third step, we try to modify x_3 . eLyMPuS answer, $\{0\}$, is incorrect because $b_3^*(x^1) = \{1\}$. Thus, we execute RECURSIVELL (Pseudocode 2, line 7), and $C = \text{Diff}(00001100, x^1) \setminus x_g = \{x_2, x_5, x_8\}$ (Pseudocode 3, line 3). At least one variable in C depends on x_3 . We randomly split C into two subsets (Pseudocode 3, line 11). Assume $gr_A = \{x_5\}$ and $gr_B = \{x_2, x_8\}$, then $x'_{s1} = 00 00 01 00$ and $b_3^*(x'_{s1}) = \{0\} \neq b_3^*(x_{s2}) = b_3^*(x^1) = \{1\}$ (lines 12 and 13). Thus, we will split $gr_B = \{x_2, x_8\}$ for solutions x'_{s1} and $x_{s2} = x^1$ (line 17). Assume that after split we receive $gr_A = \{x_2\}$ and $gr_B = \{x_8\}$, then $x'_{s1} = 01 00 01 00$ and $b_3^*(x'_{s1}) = \{1\} = b_3^*(x_{s2}) = b_3^*(x^1)$. Thus, we try to split $gr_A = \{x_2\}$ for solutions x'_{s1} and x'_{s1} (line 15). Since $|gr_A| = 1$, x_3 depends on variable in gr_A , i.e., x_2 (Pseudocode 2, lines 6-8).

In Example 3.2, the last execution of RECFORGROUP took place for x'_{s1} and x'_{s1} that differ only by x_2 and $b_3^*(x'_{s1}) \neq b_3^*(x'_{s1})$. Thus, these can be two consecutive solutions x_{i-1}, x_i concerned in Theorem 2.

Here, we focus on symmetrical dependencies. However, the above procedure allows for directional dependency discovery. Such a feature is beneficial because every symmetrical dependency is built from two directional ones. For problems with directional dependencies, discovering them enables constructing more precise eG , which may lead to a significant increase in effectiveness [17].

Pseudocodes 2 and 3 aim to present the algorithm idea rather than reflect the actual source code. The fitness-based cost of PARTIALCOMPARISON is 0 if *Verify* = false or 1 otherwise. In RECURSIVELL, the cost of a single RECFORGROUP execution is 2 because we only have to compute $f(x'_{s1})$ and $f(x'_{s1}^g)$. Thus, the FFE cost of RECURSIVELL equals $2 \lceil \log_2(|C|) \rceil$.

3.3 First Improvement Hill Climber with eLyMPuS and the Circuit Check

eLyMPuS is suitable for any local search procedure in which, in one step, only one variable is modified. If result correctness is verified at every partial comparison execution, the FFE cost of the local search will remain unchanged. However, eLyMPuS may discover missing dependencies (and subsequently, dependencies in the low-cost procedure presented in the prior subsection). Thus, in such a setting, eLyMPuS will support a zero-cost missing linkage detection and a low-cost linkage discovery with guaranteed output.

The above eLyMPuS setting is attractive but does not enable the FFE cost decrease, which is a typical surrogate goal. Therefore, we propose the First Improvement Hill Climber using eLyMPuS (FIHC-eLyMPuS). At the beginning of each FIHC-eLyMPuS execution, we decide if eLyMPuS will verify missing linkage during this execution or not (Pseudocode 4, line 2). We use a simple strategy denoted as $1/v^{th}$. After each FIHC-eLyMPuS iteration with verification, we count the number of v subsequent FIHC-eLyMPuS iterations with verification (which may be interrupted by iterations without verification) in which no missing linkage was found. Then, the next FIHC-eLyMPuS iteration with verification will be held after the next $v + 1$ iterations without verification. Thus, if eG is complete, then the verification will be held more and more rarely, which decreases the overall optimization cost because eLyMPuS without verification is free in terms of FFE. The $1/v^{th}$ strategy ensures that verification is held sometimes, which is important in black-box scenarios.

Pseudocode 4 First Improvement Hill Climber using eLyMPuS

```

1: function FIHC-eLyMPuS( $\mathbf{x}_{chk}$ ,  $eLyMPuS$ )
2:    $Verify \leftarrow eLyMPuSVerifyStrategy()$ ;
3:    $Mods(\mathbf{x}, i, b_i^*) \leftarrow \text{empty}$ ;
4:    $\pi_n \leftarrow \text{GenerateRandomPermutation}(1, n)$ ;
5:   repeat
6:      $modified \leftarrow \text{false}$ ;
7:     for each  $i$  in  $\pi_n$  do
8:        $b_i^* \leftarrow \text{PartialComparison}(x_i, \mathbf{x}_{chk}, eLyMPuS, Verify)$ ;
9:       if  $|b_i^*| = 1$  and  $b_i^* \neq \mathbf{x}_{chk}[i]$  then
10:         $Mods \leftarrow Mods + (\mathbf{x}_{chk}, i, b_i^*)$ ;
11:         $\mathbf{x}_{chk}[i] \leftarrow b_i^*$ ;
12:         $modified \leftarrow \text{true}$ ;
13:        if  $\text{CheckCircuits}(Mods, eLyMPuS) = \text{true}$  then
14:           $Mods(\mathbf{x}, i, b_i^*) \leftarrow \text{empty}$ ;
15:   until  $modified = \text{true}$ 
16:   return  $\mathbf{x}_{chk}$ ;

```

Pseudocode 5 Circuit-based guaranteed to detect missing linkage

```

1: function CHECKCIRCUITS( $Mods(\mathbf{x}, i, b_i^*), eLyMPuS$ )
2:    $LastSolModTripple \leftarrow Mods[|Mods|]$ ;
3:   for  $mod = 1$  to  $|Mods| - 1$  do
4:     if  $LastSolModTripple.x = Mods[mod].x$  then
5:       ▶ Entering this block we are guaranteed to find missing linkage
6:       and one missing dependency
7:       for  $j = mod$  to  $|Mods| - 1$  do
8:          $g \leftarrow Mods[j].i$ ;
9:          $b_g^* \leftarrow \text{Compute\_}b_g^*(x_g, Mods[j].x)$ ; ▶ Pseudocode 2
10:        if  $b_g^* \neq Mods[j].b_i^*$  then
11:           $CircMod \leftarrow \text{FindPair}(x_g, Mods[j].x, eLyMPuS)$ ;
12:           $\text{RecursiveLL}(Mods[j].x, CircMod.x, x_g, eLyMPuS)$ ;
13:           $\text{AddNewPair}(eLyMPuS.x_gPairs(\mathbf{x}, b_g^*), \text{Pair}(Mods[j].x, b_g^*))$ ;
14:          return  $\text{true}$ ;
15:   return  $\text{false}$ ;

```

In some cases, FIHC-eLyMPuS can detect missing linkage as a part of its procedure without verification. Let $\mathcal{T} = \{\mathbf{x}_0, \dots, \mathbf{x}_{|T|}\}$ be a sequence of solutions yielded during FIHC-eLyMPuS execution in subsequent steps (Pseudocode 4, lines 3 and 10). If $\mathbf{x}_{|T|} = \mathbf{x}_t$ where $t \in \{0, \dots, |T| - 1\}$, then at least one move between $\mathbf{x}_t$ and $\mathbf{x}_{|T|}$ was not an improving one, i.e., $f(\mathbf{x}_w) \geq f(\mathbf{x}_{w+1})$, where $w \in \{t, \dots, |T| - 1\}$. Thus, at least one $b_{g(w)}^*(\mathbf{x}_w) \neq eLyMPuS(x_{g(w)}, eG, \mathbf{x}_w)$, where $x_{g(w)}$ is the variable flipped in $\mathbf{x}_w$ to yield $\mathbf{x}_{w+1}$. Therefore, after each step found improving, CHECKCIRCUITS is executed (line 13).

CHECKCIRCUITS (Pseudocode 5) checks if $\mathbf{x}_{|T|} = \mathbf{x}_t$. If so, then checks subsequent $b_{g(w)}^*(\mathbf{x}_w)$ until finding the incorrect move (lines 5-8) and executes RECURSIVELL for it (line 10).

Example 3.3. Consider $f(x_1, \dots, x_8)$, $\mathbf{x}$ and eLyMPuS (Table 3) from Example 3.2. We use partial comparisons to optimize $\mathbf{x} = [11\ 10\ 01\ 01]$, but we do not verify the correctness of the partial comparison, i.e., $Verify = \text{false}$. Genes are considered in the gene order. In the first iteration, FIHC-eLyMPuS will flip x_1 , x_3 , and x_4 (for x_2 , x_5 , and x_6 the lists of $(\mathbf{x}, b_g^*(\mathbf{x}))$ pairs will be supplemented). In the second iteration, FIHC-eLyMPuS will flip x_3 and x_4 . Thus, $\mathcal{T} = \{\mathbf{x}_0 = 11\ 10\ 01\ 01, \mathbf{x}_1 = 01\ 10\ 01\ 01, \mathbf{x}_2 = 01\ 00\ 01\ 01, \mathbf{x}_3 =$

$01\ 01\ 01\ 01, \mathbf{x}_4 = 01\ 11\ 01\ 01, \mathbf{x}_5 = 01\ 10\ 01\ 01\}$, $\mathbf{x}_1 = \mathbf{x}_5$ and one of the modifications for x_1 , x_2 , x_3 , or x_4 was not an improving move. Indeed, $f(\mathbf{x}_1) > f(\mathbf{x}_2 = \mathbf{x}_1^3)$ and RECURSIVELL will discover a dependency for x_3 and supplement $eLyMPuS.eG$.

Detecting circuit in $\mathcal{T}$ does not yield FFE. If it is found, eLyMPuS is guaranteed to detect missing linkage in no more than $|\mathcal{T}|$ steps (and FFE) and discover one missing dependency using RECURSIVELL.

4 Recursive linkage learning via Partition Crossover in black-box scenarios

Originally, PX was proposed to use G_{NL} in gray-box scenarios [25]. Recently, it was shown that PX applies to eG_{NM} and black-box optimization as well [18]. In such a setting, if a given PX mask is complete, i.e., it equals one of the masks that would be yielded for G_{NM} , then $(f(\mathbf{x}_{s1})R_{inef}(\mathbf{x}_{o1}) \text{ and } f(\mathbf{x}_{s2})R_{inef}(\mathbf{x}_{o2}))$ or $(f(\mathbf{x}_{s1}) = f(\mathbf{x}_{o1}) \text{ and } f(\mathbf{x}_{s2}) = f(\mathbf{x}_{o2}))$, where $R_{ine} = \{>, <\}$, $\mathbf{x}_{s1}$ and $\mathbf{x}_{s2}$ are the mixed solutions, $\mathbf{x}_{o1} = \mathbf{x}_{s1} \xleftarrow{mPX} \mathbf{x}_{s2}$, $\mathbf{x}_{o2} = \mathbf{x}_{s2} \xleftarrow{mPX} \mathbf{x}_{s1}$, and mPX is a PX mask. Otherwise, the mask is incomplete, and we are *guaranteed* that at least one missing dependency exists between a gene from mPX and the rest of the genotype [18]. In the original proposition, after such missing linkage detection, all pairs joining each gene from mPX and outside of it are the subject to the non-monotonicity check [18]. Such a procedure is expensive, and it does not guarantee any output. We propose *PX with Recursive Linkage Learning* (PXrLL) that guarantees finding x_g on which (the incomplete) mPX depends in $\log_2(|\text{Diff}(\mathbf{x}_{s1}, \mathbf{x}_{s2}) \setminus mPX|)$ steps. We also propose a recursive procedure to find which gene from mPX depends on x_g but it does not guarantee to find the dependency. Since the procedure of PXrLL is based on the same ideas as presented in Section 3, its detailed presentation, pseudocodes and examples can be found in Section S-III (supplementary material).

5 Pyramid-based Optimizer using eLyMPuS

FIHC-eLyMPuS can be utilized in any optimizer as a local search algorithm. We introduce it to P3-FIHCwLL and LT-GOMEA-FIHCwLL [19] where it replaces FIHCwLL. However, to fully utilize the potential of eLyMPuS and other proposed mechanisms, we propose the Optimizer using eLyMPuS (OLYMPuS). OLYMPuS is a parameterless optimizer that uses a pyramid-like population management, PXrLL and the ILS-like solution improvement step supported by eLyMPuS. Below, we refer to Pseudocode S-3 (supplementary material) that presents the details of OLYMPuS.

In each iteration, PXLINKDISCOVERY is executed in which two individuals are created randomly and mixed using PX and eG . The only purpose of this operation is linkage discovery. Note that the cost of this operation (4 FFE) is negligible but sometimes it may discover dependencies.

In the main part of each iteration, OLYMPuS randomly creates a new individual $\mathbf{x}_{climb}$, optimizes it with FIHC-eLyMPuS, and inserts it into the first level of the pyramid (Pseudocode S-3, lines 6-8). Then, $\mathbf{x}_{climb}$ is the subject of the ILS-like optimization step, i.e., for each x_g (considered in a random order), genes marked by $C_{eG}^g \cup x_g$ are randomly reinitialized, FIHC-eLyMPuS optimizes the perturbed $\mathbf{x}'_{climb}$ (lines 15-16). If the fitness of the resulting $\mathbf{x}'_{climb}$ is not worse than for $\mathbf{x}_{climb}$, it replaces $\mathbf{x}_{climb}$. If ILSLIKEOPT

improves x_{climb} , its improved copy is added to the next level of the pyramid (line 19). Finally, x_{climb} is mixed with subsequent levels of the pyramid using PXRLL (line 22).

6 Experiments

6.1 Experiment setup and considered optimizers

OLyMPuS is an optimizer designed to leverage eLyMPuS's advantages. Additionally, to verify the effectiveness of eLyMPuS, we introduce it into P3-FIHCwLL and LT-GOMEA-FIHCwLL [19] by replacing the FIHCwLL mechanism with eLyMPuS. Therefore, we consider P3-FIHCwLL, LT-GOMEA-FIHCwLL, and their original versions, i.e., P3 and LT-GOMEA, as baseline optimizers. Due to space limitations, LT-GOMEA results are reported only in the supplementary material. Each experiment is repeated 30 times.

The computation budget is set on $2 \cdot 10^7$ FFE and 12 hours. If any experiment exceeds at least one of these constraints, the experiment is stopped. Such a stop condition is designed to limit the advantage of OLyMPuS, which consumes the FFE-based budget very slowly due to leveraging the advantages of eLyMPuS.

We consider a set of well-known benchmark problems that includes: bimodal-10 (Bim10), noised bimodal-10 (nBim10), and standard deceptive functions ($k = 5$, Dec5). We consider concatenations of these functions and cyclic traps where functions share variables (e.g., in Bim10o2, Bim10 functions share 2 variables with each of their neighbours). We also consider NK-fitness landscapes (nkLand) [6, 33], Ising Spin Glasses (ISG) [20], max3sat (m3s) [6, 30], mk fitness landscapes using Bim10 and Dec3 [26, 32]. The detailed definitions can be found in Section S-V (supplementary material).

The implementation of all optimizers and problems was joined in one C++ project and is available at GitHub¹. To assure that the time-based stop condition is reliable, all experiments were single-threaded and executed on the PowerEdge R6525 Dell Server AMD 2 Epyc 7H12 2.6 GHz 1TB RAM. Each experiment was repeated 30 times, and the experiment guiding software kept the number of computing processes constant and lower than the number of physical processor cores.

Optimizers capable of finding the optimal solution for larger instances were found more effective. If more optimizers scaled similarly, then the statistical significance of consumed FFE-budget was considered (depending on the instance group, we have employed the Sign test or the unpaired Wilcoxon test, both with the 5% significance level).

6.2 Main results

Fig. 1 presents two representative OLyMPuS runs. The X-axis presents the consumption of the FFE-based computation budget. In Fig. 1a, all dependencies are quickly found (the red curve reaches 1). When this happens, the number of eLyMPuS-saved FFEs increases quickly (brown curve) because no missing linkages are found, and the costs of checking them drop thanks to the $1/v^{th}$ strategy. Nevertheless, the FFE budget required to find the optimal solution was relatively large, indicating that finding G and constructing a surrogate make the optimization easier but not necessarily easy.

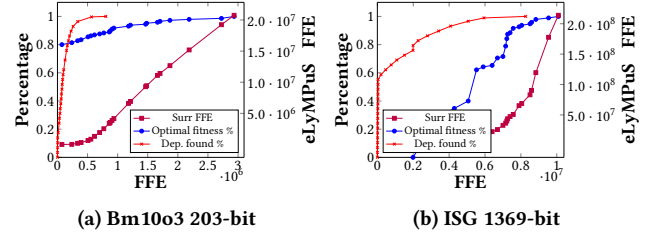


Figure 1: Representative OLyMPuS runs. The increase of the best-found solution, discovered dependency percentage, and the FFE savings brought by eLyMPuS in the experiment run (X axis – FFE of the true function).

Fig. 1b presents a run in which high-quality solutions were found before finding all dependencies. Also, the eLyMPuS-saved FFE number starts increasing before the red curve ends. After finding G , the optimal solution is found very quickly.

Table 4 presents detailed statistics of the FIHC-based costs of OLyMPuS for the chosen instance groups. For many problems, the eLyMPuS-based savings are significant (over 90% of evaluations can be handled by eLyMPuS). eLyMPuS discovers a vast majority of dependencies, and the highest cost is the missing linkage detection, i.e., verifying whether the partial comparison output returned by eLyMPuS is correct. The cost of linkage discovery is lower by one or even three orders of magnitude. Similar observations can be made for PXRLL, but its costs seem negligible when compared to FIHC-eLyMPuS. The comparison of other OLyMPuS versions is presented in Table S-V (supplementary material).

Table 4 shows that the percentage of dependencies found by PXRLL is negligible compared to eLyMPuS, leading to the conclusion that missing linkage detection and linkage discovery can be removed from PXRLL. Surprisingly, it is false. Table S-V (supplementary material) presents a detailed comparison of the standard OLyMPuS version and the version that does not perform any missing-linkage detection (and therefore does not perform any linkage discovery) in PXRLL. We consider only those instance groups for which the median percentage of the found dependencies is below 100%. In 25 out of 28 instance groups, standard OLyMPuS obtained a higher median of discovered dependencies. This difference was caused by PXRLL. For nkLand and ISG, this tiny difference caused a significant difference in the percentage of successful runs. Thus, even if the number of dependencies discovered by PXRLL and FIHC-eLyMPuS differ by several levels of magnitude, PXRLL seems to discover those dependencies that elude FIHC-eLyMPuS.

In Table 5, we show the largest considered instance solved in at least 80% of the runs by a given optimizer. If more than one optimizer reached the same instance size, then we compare the statistical significance of FFE differences. If the differences are meaningful, then the faster optimizer is considered more effective. Otherwise, more than one optimizer can be found to be the best for a given problem. OLyMPuS was among the most effective optimizers for 10 out of 16 problems and outperforms the other optimizers. Note that the second-most-effective optimizer was P3-eLyMPuS, and P3-FIHCwLL took third place. These results confirm that eLyMPuS offers a range of functionalities that can improve the

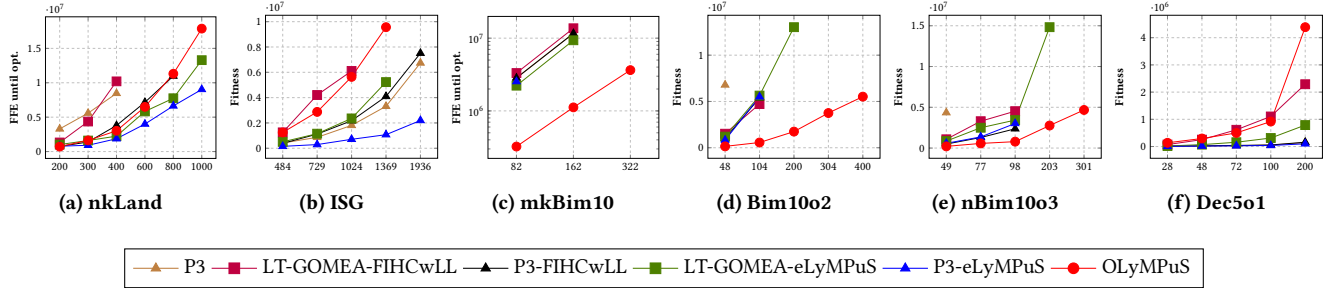
¹<https://github.com/przewooz/OLyMPuS>

Table 4: OLYMPuS – detailed results for chosen instances (median values reported). Full results: Tables S-II and S-II (supplementary material).

Problem (size)	Succ [%]	med. FFE until best Func / Surr / ratio	Dep. disc. [%]	FIHC-eLyMPuS FFE cost				PXrLL FFE cost				InitPX LL	Linkage discovery (ratio [%])			
				Total	$b'_g(x)$	Miss.	Disc.	Total	Regular	Miss.	Disc.		eLyMPuS FIHC	Circ	PXrLL Disc	InitPX Disc
nkLand (800)	100	1.1E+7 / 8.0E+7 / 88	100	1.1E+7	7.6E+5	1.0E+7	6.1E+4	1.6E+5	8.1E+4	7.8E+4	1.9E+3	7.9E+2	3984	43	5 (28)	4 (80)
ISG (1936)	60	1.9E+7 / 4.8E+8 / 96	100	1.9E+7	1.9E+5	1.9E+7	7.0E+4	3.9E+5	2.9E+5	9.7E+4	3.2E+2	8.9E+2	3768	0	5 (77)	11 (95)
m3s (75)	90	1.4E+5 / 4.7E+4 / 25	93	2.1E+5	1.3E+5	7.6E+4	5.1E+3	1.6E+1	1.4E+1	2.0E+0	0.0E+0	1.9E+2	740	0	0 (-)	5 (91)
mkDec3 (82)	100	3.6E+4 / 3.4E+4 / 49	85	3.6E+4	1.6E+3	3.3E+4	1.3E+3	1.2E+2	3.8E+1	3.6E+1	2.1E+1	1.4E+2	129	0	1 (100)	4 (78)
Bm10o1 (198)	100	1.5E+6 / 1.2E+7 / 89	100	8.6E+5	4.0E+5	4.5E+5	1.2E+4	6.6E+5	4.1E+5	2.4E+5	8.8E+1	1.3E+3	981	1	4 (100)	5 (83)
nBm10o3 (203)	100	2.8E+6 / 1.7E+7 / 86	100	2.0E+6	1.4E+6	6.3E+5	1.4E+4	7.7E+5	4.3E+5	3.3E+5	1.9E+3	1.7E+3	1186	14	19 (88)	13 (100)

Table 5: General comparison. The largest problem size solved with success rate above 80% (median FFE until finding the best solution reported). Extended results: Tables S-VI and S-VII (supplementary material).

	OLYMPuS			P3			P3-FIHCwLL			P3-eLyMPuS			LT-GOMEA-FIHCwLL			LT-GOMEA-eLyMPuS		
	Size	Succ	FFE	Size	Succ	FFE	Size	Succ	FFE	Size	Succ	FFE	Size	Succ	FFE	Size	Succ	FFE
nkLand	1000	83	1.8E+7	400	90	8.5E+6	600	93	7.2E+6	1000	83	9.0E+6	400	80	1.0E+7	1000	93	1.3E+7
ISG	1369	100	9.6E+6	1936	100	6.7E+6	1936	97	7.5E+6	1936	100	2.2E+6	1024	97	6.1E+6	1369	93	5.2E+6
m3s	75	90	1.4E+5	100	100	1.6E+5	100	97	2.1E+5	100	97	1.8E+5	75	97	1.3E+5	75	93	9.2E+4
mkBim10	322	100	3.6E+6	N/A	N/A	N/A	162	80	1.2E+7	82	100	2.5E+6	162	97	1.4E+7	162	100	9.4E+6
mkDec3	102	100	4.5E+4	102	100	5.8E+3	102	100	6.3E+3	102	100	5.5E+3	102	100	1.2E+4	102	100	1.0E+4
Bim10	400	100	3.2E+6	50	100	1.1E+6	400	90	1.3E+7	400	95	9.4E+6	400	95	9.8E+6	400	100	1.4E+7
Bim10o1	297	100	2.9E+6	45	100	2.4E+6	99	100	3.5E+6	99	100	3.2E+6	198	97	1.1E+7	297	80	1.6E+7
Bim10o2	400	95	5.5E+6	48	95	6.8E+6	104	95	5.4E+6	104	100	5.5E+6	104	100	4.7E+6	200	90	1.3E+7
Bim10o3	301	87	4.4E+6	N/A	N/A	N/A	98	93	8.3E+6	203	97	8.2E+6	98	100	9.0E+6	98	100	4.4E+6
nBim10	400	100	2.7E+6	50	100	9.9E+5	400	100	4.3E+6	400	100	3.2E+6	400	100	9.7E+6	400	100	7.0E+6
nBim10o1	297	100	3.2E+6	45	100	2.0E+6	198	100	6.1E+6	297	80	1.3E+7	198	100	8.3E+6	297	100	9.7E+6
nBim10o2	400	90	6.1E+6	48	100	4.2E+6	200	85	9.3E+6	104	100	2.1E+6	200	100	9.3E+6	304	85	1.2E+7
nBim10o3	301	83	4.7E+6	49	100	4.3E+6	98	100	2.4E+6	98	100	3.0E+6	98	100	4.5E+6	203	97	1.5E+7
Dec5	200	100	5.8E+6	200	100	6.8E+4	200	100	9.3E+4	200	100	5.8E+4	200	100	1.6E+5	200	100	1.5E+5
Dec5o1	200	100	4.4E+6	200	100	1.4E+5	200	100	1.6E+5	200	100	1.0E+5	200	100	2.3E+6	200	100	7.9E+5
Dec5o2	201	100	4.5E+6	201	100	1.6E+5	201	100	2.2E+5	201	100	1.5E+5	201	100	4.6E+6	201	100	1.5E+6
Most effective	10			5			6			8			1			2		

**Figure 2: Scalability for chosen problems**

performance of various optimizers (LT-GOMEA-eLyMPuS outperformed its FIHCwLL and base versions). Finally, these observations are confirmed by the scalability analysis presented in Fig. 2.

7 Conclusions

Here, we propose eLyMPuS, which is based on the non-monotonicity check and applies to problems that can not be modeled by perfect linear surrogates. It enables partial comparisons, i.e., comparing two solutions that differ by one variable. If eLyMPuS is trained using eG such that $eG = G$, it is guaranteed to yield a correct answer. The model-building process is not a separate step but is performed during use, and FFE-based costs are paid only when necessary, making the eLyMPuS highly flexible. Given that it is parameter-less, it may be well-suited to solving real-world problems.

eLyMPuS functionality extends beyond a typical surrogate. Indeed, within its routine, it enables low-cost missing-linkage detection and a low-cost linkage-discovery procedure that is guaranteed

to discover one missing dependency. It can also tell symmetrical and non-symmetrical dependencies. Finally, the proposed mechanisms are not limited to the binary search space and apply to integer-based solution encoding. They can also be adjusted to other search spaces, which is a future work step. However, the most important future objective is to propose a perfect monotonic surrogate without limitations, i.e., one that can compare any pair of solutions.

Acknowledgments

We would like to thank Peter Bosman for his valuable remarks. This work was supported by: Polish National Science Centre (NCN), 2022/45/B/ST6/04150 (M.M. Komarnicki, M.W. Przewozniczek); MCIN/AEI/10.13039/501100011033, PID2024-158752OB-I00, the University of Malaga, PAR 4/2023 (F. Chicano); FAPESP, #2024/15430-5 and #2024/08485-8, CNPq, #304640/2024-7 (R. Tinós).

References

- [1] Edmund Burke and Sanja Petrovic. 2002. Recent research directions in automated timetabling. *Europ. Jour. of Oper. Res.* 140 (07 2002), 266–280. doi:10.1016/S0377-2217(02)00069-3
- [2] Wei-Ming Chen, Chung-Yu Shao, Po-Chun Hsu, and Tian-Li Yu. 2012. A test problem with adjustable degrees of overlap and conflict among subproblems. In *Proceedings of the 14th Annual Conference on Genetic and Evolutionary Computation (GECCO '12)*. ACM, 257–264.
- [3] Francisco Chicano, Bilel Derbel, and Sébastien Verel. 2023. Fourier Transform-based Surrogates for Permutation Problems. In *Proceedings of the Genetic and Evolutionary Computation Conference (Lisbon, Portugal) (GECCO '23)*. Association for Computing Machinery, New York, NY, USA, 275–283. doi:10.1145/3583131.3590425
- [4] Kalyanmoy Deb, Jeffrey Horn, and David E. Goldberg. 1993. Multimodal Deceptive Functions. *Complex Systems* 7, 2 (1993).
- [5] Arkadiy Dushatskiy, Tanja Alderliesten, and Peter A. N. Bosman. 2021. A Novel Approach to Designing Surrogate-assisted Genetic Algorithms by Combining Efficient Learning of Walsh Coefficients and Dependencies. *ACM Trans. Evol. Learn. Optim.* 1, 2, Article 5 (July 2021), 23 pages. doi:10.1145/3453141
- [6] Brian W. Goldman and William F. Punch. 2014. Parameter-less Population Pyramid. In *Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation (Vancouver, BC, Canada) (GECCO '14)*. ACM, 785–792.
- [7] R. B. Heckendorn. 2002. Embedded Landscapes. *Evolutionary Computation* 10, 4 (2002), 345–369.
- [8] Shih-Huan Hsu and Tian-Li Yu. 2015. Optimization by Pairwise Linkage Detection, Incremental Linkage Set, and Restricted / Back Mixing: DSMGA-II. In *Proceedings of the 2015 Annual Conference on Genetic and Evolutionary Computation (GECCO '15)*. ACM, 519–526.
- [9] Yaochu Jin. 2011. Surrogate-assisted evolutionary computation: Recent advances and future challenges. *Swarm and Evolutionary Computation* 1, 2 (2011), 61–70. doi:10.1016/j.swevo.2011.05.001
- [10] Marcin Michal Komarnicki, Michal Witold Przewozniczek, Halina Kwasnicka, and Krzysztof Walkowiak. 2023. Incremental Recursive Ranking Grouping for Large-Scale Global Optimization. *IEEE Transactions on Evolutionary Computation* 27, 5 (2023), 1498–1513.
- [11] Marcin Michal Komarnicki, Michal Witold Przewozniczek, Renato Tinós, and Xiaodong Li. 2024. Overlapping Cooperative Co-Evolution for Overlapping Large-Scale Global Optimization Problems. In *Proceedings of the Genetic and Evolutionary Computation Conference (Melbourne, VIC, Australia) (GECCO '24)*. Association for Computing Machinery, New York, NY, USA, 665–673. doi:10.1145/3638529.3654171
- [12] Florian Lepître, Sébastien Verel, Cyril Fonlupt, and Virginie Marion. 2019. Walsh functions as surrogate model for pseudo-boolean optimization problems. In *Proceedings of the Genetic and Evolutionary Computation Conference (Prague, Czech Republic) (GECCO '19)*. Association for Computing Machinery, New York, NY, USA, 303–311. doi:10.1145/3321707.3321800
- [13] Xiaoliang Ma, Zhitao Huang, Xiaodong Li, Lei Wang, Yutao Qi, and Zexuan Zhu. 2022. Merged Differential Grouping for Large-Scale Global Optimization. *IEEE Transactions on Evolutionary Computation* 26, 6 (2022), 1439–1451. doi:10.1109/TEVC.2022.3144684
- [14] M. Munetomo and D.E. Goldberg. 1999. A genetic algorithm using linkage identification by nonlinearity check. In *IEEE SMC '99 Conference Proceedings. 1999 IEEE International Conference on Systems, Man, and Cybernetics (Cat. No.99CH37028)*, Vol. 1. 595–600 vol.1. doi:10.1109/ICSMC.1999.814159
- [15] Masaharu Munetomo and David E. Goldberg. 1999. Linkage identification by non-monotonicity detection for overlapping functions. *Evol. Comput.* 7, 4 (dec 1999), 377–398. doi:10.1162/evco.1999.7.4.377
- [16] Michal Witold Przewozniczek, Francisco Chicano, Renato Tinós, Jakub Nalepa, Bogdan Rusczyk, and Agata Wijata. 2025. On Revealing the Hidden Problem Structure in Real-World and Theoretical Problems Using Walsh Coefficient Influence. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO '25)*. ACM, 295–303.
- [17] Michal Witold Przewozniczek, Bartosz Frej, and Marcin Michal Komarnicki. 2025. From Direct to Directional Variable Dependencies—Nonsymmetrical Dependencies Discovery in Real-World and Theoretical Problems. *IEEE Transactions on Evolutionary Computation* 29, 2 (2025), 490–504. doi:10.1109/TEVC.2024.3496193
- [18] Michal W. Przewozniczek, Renato Tinós, Bartosz Frej, and Marcin M. Komarnicki. 2022. On Turning Black - into Dark Gray-Optimization with the Direct Empirical Linkage Discovery and Partition Crossover. In *Proceedings of the Genetic and Evolutionary Computation Conference (Boston, Massachusetts) (GECCO '22)*. Association for Computing Machinery, New York, NY, USA, 269–277. doi:10.1145/3512290.3528734
- [19] Michal Witold Przewozniczek, Renato Tinós, and Marcin Michal Komarnicki. 2023. First Improvement Hill Climber with Linkage Learning – on Introducing Dark Gray-Box Optimization into Statistical Linkage Learning Genetic Algorithms. In *Proceedings of the Genetic and Evolutionary Computation Conference (Lisbon, Portugal) (GECCO '23)*. ACM, 946–954.
- [20] Lawrence Saul and Mehran Kardar. 1994. The 2D±J Ising spin glass: exact partition functions in polynomial time. *Nuclear Physics B* 432, 3 (1994), 641–667.
- [21] Yuan Sun, Xiaodong Li, Andreas Ernst, and Mohammad Nabi Omidvar. 2019. Decomposition for Large-scale Optimization Problems with Overlapping Components. In *2019 IEEE Congress on Evolutionary Computation (CEC)*. 326–333. doi:10.1109/CEC.2019.8790204
- [22] Dirk Thierens. 2010. The Linkage Tree Genetic Algorithm. In *Parallel Problem Solving from Nature, PPSN XI: 11th International Conference, Kraków, Poland, September 11–15, 2010, Proceedings, Part I*. 264–273.
- [23] Dirk Thierens and Peter Bosman. 2012. Predetermined versus Learned Linkage Models. In *Proceedings of the 14th Annual Conference on Genetic and Evolutionary Computation (Philadelphia, Pennsylvania, USA) (GECCO '12)*. Association for Computing Machinery, New York, NY, USA, 289–296.
- [24] Renato Tinós, Michal W. Przewozniczek, and Darrell Whitley. 2022. Iterated Local Search with Perturbation Based on Variables Interaction for Pseudo-Boolean Optimization. In *Proceedings of the Genetic and Evolutionary Computation Conference (Boston, Massachusetts) (GECCO '22)*. ACM, 296–304.
- [25] Renato Tinós, Darrell Whitley, and Francisco Chicano. 2015. Partition Crossover for Pseudo-Boolean Optimization. In *Proceedings of the 2015 ACM Conference on Foundations of Genetic Algorithms XIII (Aberystwyth, United Kingdom) (FOGA '15)*. Association for Computing Machinery, New York, NY, USA, 137–149. doi:10.1145/2725494.2725497
- [26] Tobias van Driessel and Dirk Thierens. 2021. Benchmark Generator for TD Mk Landscapes. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*. Association for Computing Machinery, New York, NY, USA, 1227–1233.
- [27] Valentin Vendi, Sébastien Verel, and Cyril Fonlupt. 2024. Sparse Surrogate Model for Optimization: Example of the Bus Stops Spacing Problem. In *Evolutionary Computation in Combinatorial Optimization*, Thomas Stützle and Markus Wagner (Eds.). Springer Nature Switzerland, Cham, 16–32.
- [28] Sébastien Verel, Bilel Derbel, Arnaud Liefvooghe, Hernán Aguirre, and Kiyoshi Tanaka. 2018. A Surrogate Model Based on Walsh Decomposition for Pseudo-Boolean Functions. In *Parallel Problem Solving from Nature – PPSN XV*, Anne Auger, Carlos M. Fonseca, Nuno Lourenço, Penousal Machado, Luís Paquete, and Darrell Whitley (Eds.). Springer International Publishing, Cham, 181–193.
- [29] D. Whitley. 2019. Next generation genetic algorithms: a user's guide and tutorial. In *Handbook of Metaheuristics*. Springer, 245–274.
- [30] Darrell Whitley, Hernan Aguirre, and Andrew Sutton. 2020. Understanding Transforms of Pseudo-Boolean Functions. In *Proceedings of the 2020 Genetic and Evolutionary Computation Conference (Cancún, Mexico) (GECCO '20)*. Association for Computing Machinery, New York, NY, USA, 760–768.
- [31] Darrell Whitley, Gabriela Ochoa, and Francisco Chicano. 2025. How Partition Crossover Exposes Parallel Lattices and the Fractal Structure of k-Bounded Functions. In *Proceedings of the Genetic and Evolutionary Computation Conference (NH Malaga Hotel, Malaga, Spain) (GECCO '25)*. ACM, 836–844.
- [32] L. D. Whitley, F. Chicano, and B. W. Goldman. 2016. Gray Box Optimization for Mk Landscapes (NK Landscapes and MAX-kSAT). *Evolutionary Computation* 24, 3 (Sep. 2016), 491–519.
- [33] A.H. Wright, R.K. Thompson, and Jian Zhang. 2000. The computational complexity of N-K fitness functions. *IEEE Transactions on Evolutionary Computation* 4, 4 (2000), 373–379.
- [34] Kaveh Zadeh, Fatemeh Harsej, Mahboubeh Sadeghpour, and Mohammad Molani Aghdam. 2022. Designing a multi-echelon closed-loop supply chain with disruption in the distribution centers under uncertainty. *Journal of Industrial and Management Optimization* 19 (01 2022). doi:10.3934/jimo.2022057